

Rangsorolás

Kurtyák Dániel

Rangsorolás

- MDN - Dokumentáció
- a stílusokról el kell dönteni, hogy mi alapján rangsoroljuk őket, ha azok ütköznek
- ez lehet öröklési és szűrési tényezők által

Öröklődés

- MDN - Dokumentáció
- azoknak a tulajdonságoknak, melyeknek nincs alapértelmezett értéke, öröklődnek a leszármazott elemekre
 - pl.: `color` öröklődik, `border` nem
 - az MDN írja, hogy egy elem öröklődik-e vagy sem
- ha egy tulajdonság öröklődik, de mi nem szeretnénk, hogy öröklődjön, az `initial` értéket kell az adott tulajdonságra beállítanunk

```
div {  
  color: blue;  
  p {  
    color: initial;  
  }  
}
```

Öröklődés

- ha azt szeretnénk, hogy egy nem öröklődő tulajdonság öröklődjön, akkor az `inherit` értéket kell az adott tulajdonságra beállítanunk

```
div {  
  color: 2px solid blue;  
  p {  
    border: inherit;  
  }  
}
```

Szűkítés

- a stílusoknak a pontossága lesz meghatározó
- a szűkebb (pontosabb) a stílusok súlya nagyobb, mint a tágabbaké, ezért azt fogja a böngésző használni

Stílusok rangsorolása

- böngésző stílus
 - alapértelmezett megjelenési mód a böngészőbe beépítve
- külső stílus
 - belinkelt külső CSS fájl
- beágyazott stílus
 - `<style>` tag formázásai
 - elsősorban a külső stílus támogatott
- szövegekőzi (inline) stílus
 - a HTML elem style attribútumában megadott tulajdonságok
 - nem ajánlott
 - tudatosan kell használni, mert minden más formázást felülír

Specificity érték számítása

- MDN - Dokumentáció
- a böngésző eldönti egy szelektorról, hogy mennyire specifikus az adott elemre
- a legnagyobb specifikusságú szelektor fog rá érvényesülni
- amennyiben a specificity értéke két szelektornak egyezik, akkor mindig a legutolsó fog érvényesülni

Specificity

A kiértékelés során a szelektort az alábbiak alapján értékeljük ki:

- **a** (*ID column*): az azonosítókijelölők (pl.: `#example`) száma, minden egyes azonosító után a specificity súlyához adjunk `1-0-0` értéket.
- **b** (*CLASS column*): az osztálykijelölők (pl.: `.myClass`), az attribútum kijelölők (pl.: `[type="text"]`) és a pszeudo osztályok (pl.: `:hover` , `:nth-of-type(3n)`) száma, minden egyes ilyen után a specificity súlyához adjunk `0-1-0` értéket.
- **c** (*TYPE column*): az elemkijelölők (pl.: `div` , `p`) és a pszeudo elemek (pl.: `::before`) száma, minden egyes ilyen után a specificity súlyához adjunk `0-0-1` értéket.
- Az univerzális szelektor `(*)` és a `:where()` pszeudo osztály, mint az egyes kombinátorok a specificity számításánál figyelemmen kívül maradnak.

Specificity

Az így kapott értéket három oszlopba írhatjuk fel, mindig balról → jobbra kell az oszlopot össze vetni

- a az egyik oszlop értéke nagyobb, mint a másiké, akkor annak nagyobb a specificity értéke
- a egyezik, akkor a következő oszlopot kell összevetni.
- az következő dián növekvő sorrendben vannak a specificity értékek

Specificity

Szelektor	a (ID)	b (CLASS)	c (TYPE)
p	0	0	1
div + p	0	0	2
div.bg-lime-400	0	1	1
p span.text-white	0	1	2
#name	1	0	0
#email::before	1	0	1
#password:invalid	1	1	0

Példák az MDN-ről

```
#myElement {  
  color: green; /* 1-0-0 - WINS!! */  
}  
.bodyClass .sectionClass .parentClass [id="myElement"] {  
  color: yellow; /* 0-4-0 */  
}
```

```
#myElement {  
  color: yellow; /* 1-0-0 */  
}  
#myApp [id="myElement"] {  
  color: green; /* 1-1-0 - WINS!! */  
}
```

```
input.myClass {  
  color: yellow; /* 0-1-1 */  
}  
:root input {  
  color: green; /* 0-1-1 WINS because it comes later */  
}
```

Példák az MDN-ről

```
p {  
  /* 0-0-1 */  
}  
:is(p) {  
  /* 0-0-1 */  
}  
  
h2:nth-last-of-type(n + 2) {  
  /* 0-1-1 */  
}  
h2:has(~ h2) {  
  /* 0-0-2 */  
}  
  
div.outer p {  
  /* 0-1-2 */  
}  
div:not(.inner) p {  
  /* 0-1-2 */  
}
```